

Chapter 1: Basic Gray-Level Image Processing

Contents

1.1 Basic gray-level image processing

1.2 Basic binary image processing

1.1 Basic gray-level image processing

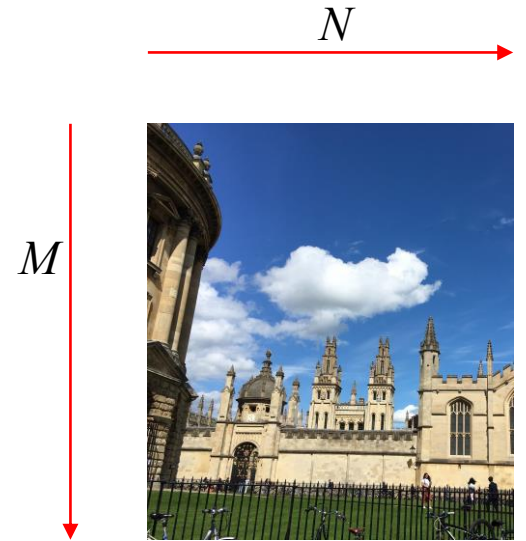
- Point Operations:
 - Applied to individual pixels without considering spatial information (neighboring pixels).
 - Defined solely by pixel intensity.
 - Key tool: Image histogram.
- Arithmetic Operations:
 - Pointwise operations between images with the same spatial dimensions.
 - Used for noise reduction, motion detection, and other specific purposes.
- Geometric Operations:
 - Modify images by changing the spatial position of pixels (e.g., translation, rotation).
 - Focus on basic transformations, excluding complex effects handled by computer graphics.

1.1 Basic gray-level image processing - Image Histograms

- Definition of Image Histogram H_f :
 - Histogram H_f represents the frequency of each gray level in an image f .
 - It is a one-dimensional function with the domain $\{0 \dots K-1\}$ and range extending to the number of pixels MN .
- Histogram Computation:

$$H_f(k) = J$$

where J is the occurrences of gray level k in image f



1.1 Basic gray-level image processing - Image Histograms

- Loss of information:
 - Histogram reduces dimensionality, losing spatial information.
 - Multiple images can share the same histogram despite having different spatial structures.
- Application: calculate Average Optical Density (AOD) which is useful and simple meter for estimating the center of an image's gray-level distribution.

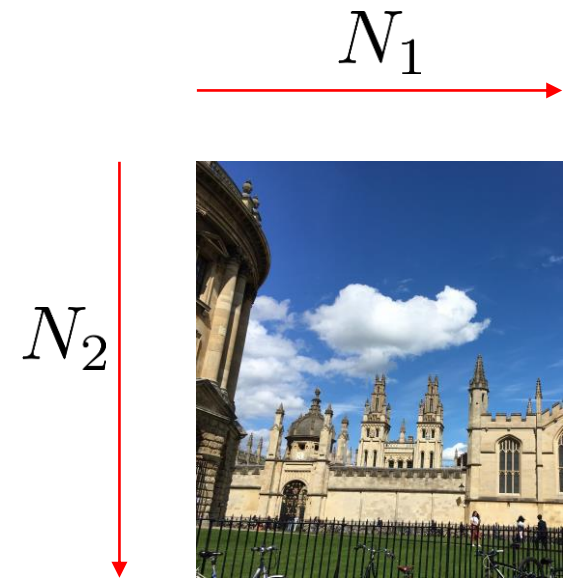
$$\text{AOD}(f) = \frac{1}{MN} \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} f(n, m) = \frac{1}{MN} \sum_{k=0}^{K-1} k H_f(k)$$

1.1 Basic gray-level image processing - Linear Point Operations on Images

- Point Operations: a function applied identically to every pixel in an image creating a new, modified image $g(\mathbf{n})$ where $\mathbf{n} = (n_1, n_2)$

$$g(\mathbf{n}) = h[f(\mathbf{n})]$$

- The limitations and scope of point operations:
 - No spatial information is utilized
 - No changes in the spatial relationships between pixels
 - Focus on gray-level manipulation



1.1 Basic gray-level image processing - Linear Point Operations on Images

- Linear Point Operations: each pixel of new image is a linear function of each pixel of input image

$$g(\mathbf{n}) = Pf(\mathbf{n}) + L \qquad 0 \leq g(\mathbf{n}) \leq K - 1$$

a gray-level additive offset L and a gray-level multiplicative scaling P of the image f

- Additive Image Offset: used to calibrate images to a given average brightness level

$$g_m(\mathbf{n}) = f_m(\mathbf{n}) - L_m + K/2 \text{ where } L_m = \text{AOD}(f_m)$$

1.1 Basic gray-level image processing - Linear Point Operations on Images

- Multiplicative Image Scaling

$$g(\mathbf{n}) = \text{INT}[Pf(\mathbf{n}) + 0.5]$$

- In terms of the image histogram

$$H_g\{\text{INT}[Pf(\mathbf{n}) + 0.5]\} = H_f(k)$$

- $P > 1$, then the gray levels of g will cover a broader range than those of f (stretches)
- $P < 1$, then the gray levels of g will have a narrower gray-level distribution than f (compresses)

1.1 Basic gray-level image processing - Linear Point Operations on Images

- Full-Scale Histogram Stretch (FSHS)

$$g(\mathbf{n}) = \text{FSHS}(f) = \left(\frac{K - 1}{B - A} \right) [f(\mathbf{n}) - A]$$

where $A = \min_{\mathbf{n}} \{f(\mathbf{n})\}$ and $B = \max_{\mathbf{n}} \{f(\mathbf{n})\}$, the histogram's domain $\{0 \dots K-1\}$

1.1 Basic gray-level image processing - Nonlinear Point Operations on Images

- Logarithmic Point Operations

$$g(\mathbf{n}) = \text{FSHS}\{\log[1 + f(\mathbf{n})]\}$$

- Histogram Equalization

- Normalized image histogram $p_f(k) = \frac{1}{MN} H_f(k)$

- Cumulative normalized image histogram

$$P_f(r) = \sum_{k=0}^r p_f(k) \text{ where } r = 0, 1, \dots, K - 1$$

- The result

$$g(\mathbf{n}) = P_f[f(\mathbf{n})]$$

1.1 Basic gray-level image processing - Arithmetic Operations Between Images

- Arithmetic operations are defined on multiple images: n images with size of $N \times M$.
- The basic operations are pointwise image addition/subtraction and pointwise image multiplication/division.
 - The sum of n images is given by

$$f_1 + f_2 + \cdots + f_n = \sum_{m=1}^n f_m$$

- The *pointwise product* of the n images

$$f_1 \otimes f_2 \otimes \cdots \otimes f_n = \prod_{m=1}^n f_m$$

1.1 Basic gray-level image processing - Arithmetic Operations Between Images

Image-Averaging for Noise Reduction

- Images that occur in practical applications invariably suffer from random degradations that are collectively referred to as *noise*.

$$f_m = g + q_m$$

- Several reasons:
 - Radiation scatter from the surface before the image is sensed;
 - Electrical noise in the sensor or camera;
 - Channel noise as the image is transmitted over a communication channel;
 - Bit errors after the image is digitized

1.1 Basic gray-level image processing - Arithmetic Operations Between Images

Image-Averaging for Noise Reduction

- Averaging the multiple noisy images (the noise is zero mean)

$$\begin{aligned}\left(\frac{1}{n}\right) \sum_{m=1}^n f_m &= \left(\frac{1}{n}\right) \sum_{m=1}^n (g + q_m) \\ &= \left(\frac{1}{n}\right) \sum_{m=1}^n g + \left(\frac{1}{n}\right) \sum_{m=1}^n q_m \\ &\approx g\end{aligned}$$

1.1 Basic gray-level image processing - Geometric Image Operations

- Definition: Geometric image operations modify the spatial positions and relationships of pixels.
- Characteristics:
 - Unlike point operations, they do not alter gray level values.
 - These operations can be computationally intensive, especially with video sequences.
 - Often involve complex algorithms.
- Applications:
 - correcting or improving real-world images.
 - Complex geometric operations are less commonly used.

1.1 Basic gray-level image processing - Geometric Image Operations

- Geometric vs. Point Operations
 - Geometric operations affect spatial relationships, while point operations affect pixel values.
 - Usage depends on the field and the intended purpose of the image processing.
- A geometric image operation generally requires two steps
 - a spatial mapping of the coordinates of an original image f to define a new image g
$$g(\mathbf{n}) = f(\mathbf{n}') = f[a(\mathbf{n})]$$
 - interpolation of non-integer coordinates to integer values, so that g can be expressed in a standard row-column format.

1.1 Basic gray-level image processing - Geometric Image Operations

Nearest Neighbor Interpolation

- The geometrically transformed coordinates are mapped to the nearest integer coordinates of f

$$g(\mathbf{n}) = f\{\text{INT}[a_1(n_1, n_2) + 0.5], \text{INT}[a_2(n_1, n_2) + 0.5]\}$$

where $\text{INT}[R]$ denotes the nearest integer that is less than or equal to R

1.2 Basic binary image processing

- A digital image with only two possible gray levels (0 and 1).
- Fast processing using logical (Boolean) operators.
- One bit per pixel, resulting in two possible values: 0 (false) and 1 (true).
- Used to represent essential information abstracted from gray-level images.
- Applications:
 - Object location and boundaries.
 - Presence or absence of specific image properties.

1.2 Basic binary image processing

- Display:
 - Convention: Displayed using black (0) and white (255) in grayscale systems.
 - Note on conventions: "1" usually represents black, but this can be reversed.
- Generation of Binary Images
 - Created from gray-level images for simplified processing or printing.
 - Direct output from sensors, e.g., for handwritten or line drawing images.

1.2 Basic binary image processing – Image thresholding

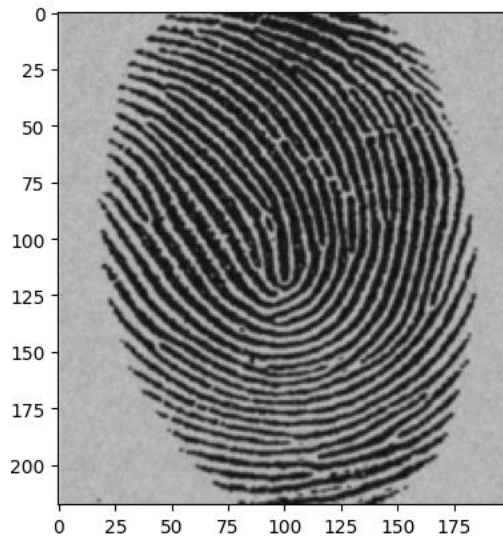
- A binary image is obtained from a gray-level image by abstracting essential information.
- Significant reduction in storage space, but potential loss of information
=> When done carefully, enhances subsequent image processing, analysis, or interpretation.
- Thresholding as the simplest form of abstraction: Converts a gray-level image into a binary image by comparing each pixel's intensity to a threshold value.

1.2 Basic binary image processing – Image thresholding

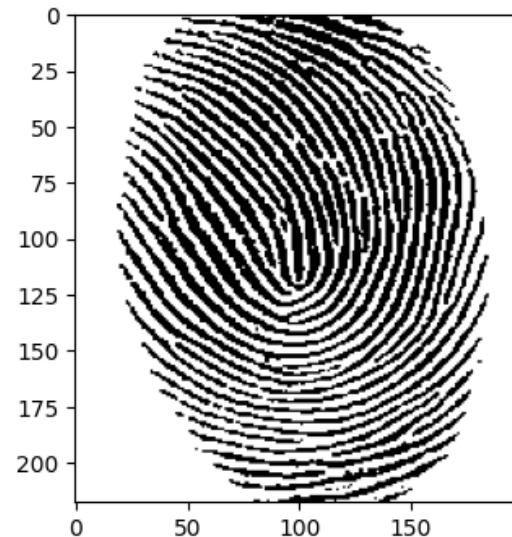
- Mechanism

$$g(\mathbf{n}) = \begin{cases} \text{"0"} & \text{if } f(\mathbf{n}) \geq T \\ \text{"1"} & \text{if } f(\mathbf{n}) < T \end{cases}$$

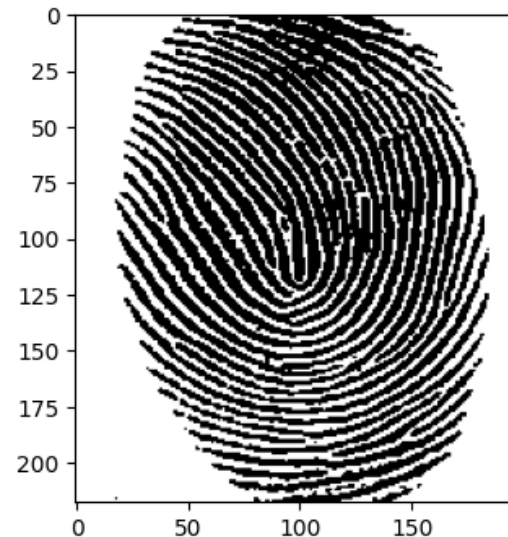
the threshold T that is used is of critical importance, since it controls the particular abstraction of information that is obtained.



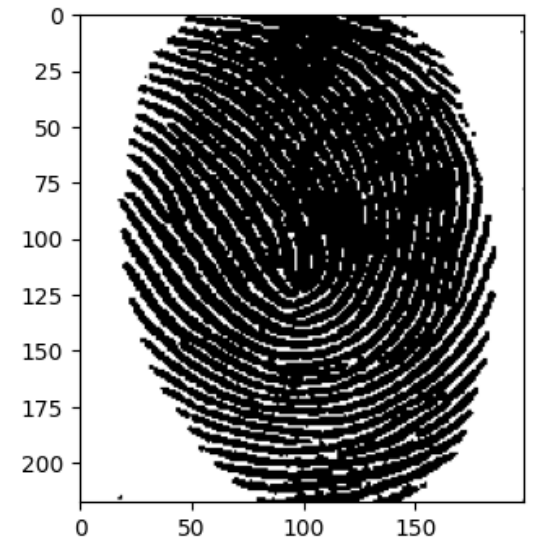
Origin



$T = 80$



$T = 120$



$T = 160$

1.2 Basic binary image processing – Image thresholding: Otsu's method

- Compute histogram and probabilities of each intensity level

- Compute initial $\omega_c(0) = \sum_{i=0}^{t-1} p(i)$ and $\mu_c(0) = \frac{\sum_{i=0}^{t-1} ip(i)}{\omega_0(t)}$
where $c = \{0, 1\}$

- Step through all possible thresholds $t = 1, \dots$ maximum intensity

- Update $\omega_c(0)$ and $\mu_c(0)$
- Compute weighted sum of variances of the two classes

$$\sigma_{\omega}^2(t) = \omega_0(t)\sigma_0^2(t) + \omega_1(t)\sigma_1^2(t) = \omega_0(t)\omega_1(t)[\mu_0(t) - \mu_1(t)]^2$$

- Desired threshold corresponds to the maximum $\sigma_{\omega}^2(t)$

1.2 Basic binary image processing – Image thresholding: Otsu's method

- Implementation using Python

1.2 Basic binary image processing – Image thresholding: Advantages

- Efficiency in Storage:
 - Reduction in required storage space by a factor -> Importance in applications with limited storage capacity or for rapid processing
- Simplification of Image Data:
 - Conversion of complex gray-level information into binary form -> Easier for subsequent processing, such as edge detection, object recognition, or pattern analysis.
- Improved Image Analysis:
 - Ability to highlight specific features of interest (e.g., separating foreground from background).
- Potential for Enhanced Interpretability:
 - easier interpretation by both human analysts and automated systems; Reduction in noise and irrelevant details.

1.2 Basic binary image processing – Image thresholding: Disadvantages

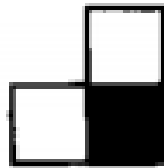
- Significant Loss of Information:
 - Binary images lose the gradient and detailed information present in gray-level images;
 - Potential negative impact on the accuracy of subsequent image analysis or interpretation
- Sensitivity to Threshold Selection:
 - The result is highly dependent on the chosen threshold value
 - Suboptimal thresholding can lead to poor segmentation and loss of important details
 - Otsu's method or adaptive thresholding, but note their complexity and limitations.
- Inability to handle Complex Images
 - not be effective for images with gradual changes in intensity or overlapping features
- Impact on Subsequent Processing
 - Potential challenges in tasks like edge detection, where binary images may lack the necessary detail.
 - Risk of oversimplification, leading to errors in object recognition or classification.

1.2 Basic binary image processing – Region Labeling

- Definition: A method to identify and label connected objects in a binary image.
- Objective
 - Identify connected groups of pixels with the same binary value.
 - Facilitate further image processing and analysis.

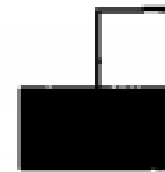
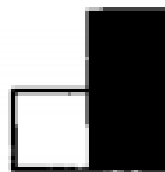
1.2 Basic binary image processing – Region Labeling

1. Given an $N \times M$ binary image f , initialize an associated $N \times M$ region label array: $r(\mathbf{n}) = "0"$ for all $\mathbf{n}$. Also initialize a region number counter: $k = 1$. Then, scanning the image from left-to-right and top-to-bottom, for every $\mathbf{n}$ do the following:
2. If $f(\mathbf{n}) = "0"$ then do nothing.
3. If $f(\mathbf{n}) = "1"$ and also $f(\mathbf{n} - (1,0)) - f(\mathbf{n} - (0,1)) = "0"$, then set $r(\mathbf{n}) = "k"$ and $k = k + 1$. In this case the left and upper neighbors of $f(\mathbf{n})$ do not belong to objects.



1.2 Basic binary image processing – Region Labeling

4. If $f(\mathbf{n})="1"$, $f(\mathbf{n}-(1,0))="1"$ and $f(\mathbf{n}-(0,1))="0"$, then set $r(\mathbf{n})=r(\mathbf{n}-(1,0))$. In this case the upper neighbor $f(\mathbf{n}-(1,0))$ belongs to the same object as $f(\mathbf{n})$.
5. If $f(\mathbf{n})="1"$, $f(\mathbf{n}-(1,0))="0"$ and $f(\mathbf{n}-(0,1))="1"$, then set $r(\mathbf{n})=r(\mathbf{n}-(0,1))$. In this case the left neighbor $f(\mathbf{n}-(1,0))$ belongs to the same object as $f(\mathbf{n})$.
6. If $f(\mathbf{n})="1"$, and $f(\mathbf{n}-(1,0)) = f(\mathbf{n}-(0,1))="1"$, then set $r(\mathbf{n})=r(\mathbf{n}-(0,1))$. If $r(\mathbf{n}-(0,1)) \neq r(\mathbf{n}-(1,0))$, then record the labels $r(\mathbf{n}-(0,1))$ and $r(\mathbf{n}-(1,0))$ as equivalent. In this case both the left and upper neighbors belong to the same object as $f(\mathbf{n})$, although they may have been labeled differently.



1.2 Basic binary image processing – Binary Image Morphology

- A class of image processing operations focused on modifying the shapes of objects within a binary image.
- Equivalent in a mathematical sense to gray-level morphologic operations.
- Altering and analyzing the structure of objects in binary images.
- Applications: Shape analysis, noise reduction, image enhancement, object detection.
- Examples: Erosion: Shrinking objects; Dilation: Expanding objects; Opening: Removing small objects; Closing: Filling small holes.

1.2 Basic binary image processing – Binary Image Morphology

- Logical Operations
 - Key Logical Operators: NOT, AND, OR, MAJ
- These logical operations form the foundation for defining and implementing morphologic transformations on binary images.

1.2 Basic binary image processing – Binary Image Morphology

- Window mechanism: a geometric rule for collecting gray levels from the vicinity of a given pixel coordinate.
- Facilitates local logical operations by defining the area over which these operations are performed.
- A morphologic operation is seen as a moving collection of empty pixels passed over the binary image.
- Operation
 - The window moves across the image, typically row-by-row, column-by-column.
 - Local logical operations are performed as the window centers over each pixel.

Summary

- Basic gray-level image processing
 - ✓ Image Histograms
 - ✓ Linear Point Operations on Images
 - ✓ Nonlinear Point Operations on Images
 - ✓ Arithmetic Operations Between Images
 - ✓ Geometric Image Operations
 - ✓ Image thresholding
- Basic binary image processing
 - ✓ Region Labeling
 - ✓ Binary Image Morphology